

청춘 일상 다이어트 총력전

개발자 : 정유운 [20231055]

발주자 : 오은찬 [2023114991]

청춘 일상 다이어트 총력전

< 1. 개요 >

(1) 프로젝트 목적

식단 및 체중을 보다 더 간편하고 세부적으로 관리하기 위해 만들어 졌으며 해당 소프트웨어를 통하여 사용자가 현재 건강 상태를 자신의 식습관과 체중을 통해 파악하고 더 나아가 식습관 관리를 통하여 건강 상태를 한 층 끌어올릴 수 있게 만들 수 있도록 한다.

청춘 일상 다이어트 총력전

< 1. 개요 >

(2) 어플리케이션 구성

사용 언어 : Python 3.14

주요 라이브러리 : Tkinter, Json

파일 구성 : 1. main.py

2. Services 폴더 (gui, operation, data manager)

3. Data 폴더 (backup 폴더, records.json, food.json)

청춘 일상 다이어트 총력전

< 1. 개요 >

(3) 주요 기능

1. 날짜별 식단 관리
2. 음식 관리
3. 식단 조회
4. 최근 체중 경향, 목표 달성 여부

청춘 일상 다이어트 총력전

< 1. 개요 >

(4) 운영 및 유지 보수

해당 어플리케이션은 GitHub를 통하여 배포하고 있으며 버그가 발생되어 수정하는 경우 새로운 버전으로 재출시 할 예정이다.

[GitHub Link] - <https://github.com/Ovie-hex/Software-Project-P1-Group-5>

2026-09-26 (토)

조회
삭제

아침

음식 관리

잡곡밥 · 55 kcal

X

시금치 · 45 kcal

X

소고기 국 · 423 kcal

X

점심

음식 관리

잡곡밥 · 55 kcal

X

잡곡밥 · 55 kcal

X

저녁

음식 관리

시금치 · 45 kcal

X

시금치 · 45 kcal

X

시금치 · 45 kcal

X

시금치 · 45 kcal

X

시금치 · 45 kcal

X

시금치 · 45 kcal

X

소고기 국 · 423 kcal

X

소고기 국 · 423 kcal

X

소고기 국 · 423 kcal

X

야식

음식 관리

잡곡밥 · 55 kcal

X

시금치 · 45 kcal

X

소고기 국 · 423 kcal

X

총 열량 : 2830 kcal

입력 체중 (kg)

66

입력 반영값 (확인 전): 66 kg

체중 반영

확인

메인으로 돌아가기

1. 날짜별 식단 관리

(1) 날짜 조회

- 조회 버튼을 눌러 식단을 관리할 날짜를 선택 할 수 있다

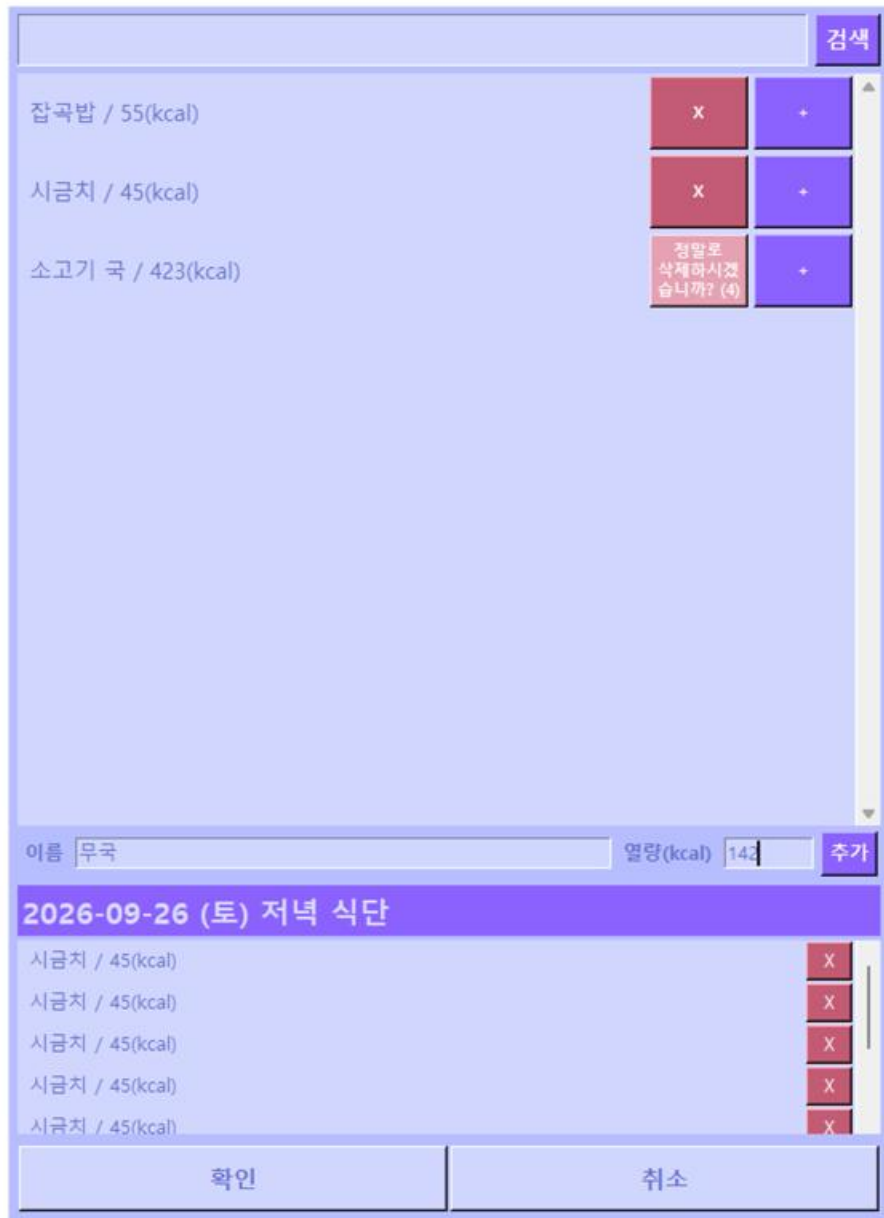
(2) 음식 관리 기능

- 각 시간대 별로 음식을 추가하고 잘못된 항목이 있는 경우 삭제가 가능하다

(3) 체중 입력

- 해당 날짜 기준 사용자의 체중을 입력한다

확인 버튼을 누르면 변경 내역이 반영되고 메인 버튼을 누르면 해당 변경 내역은 취소된다.



잡곡밥 / 55(kcal)

시금치 / 45(kcal)

소고기 국 / 423(kcal)

정말로 삭제하시겠습니까? (4)

이름 무국 열량(kcal) 142 추가

2026-09-26 (토) 저녁 식단

시금치 / 45(kcal)

시금치 / 45(kcal)

시금치 / 45(kcal)

시금치 / 45(kcal)

시금치 / 45(kcal)

확인 취소

2. 음식 관리

(1) 음식 검색

- 사용자가 문자열을 입력하고 검색 버튼을 누르면 해당 문자열과 매칭되는 음식들을 조회한다.

(2) 음식 추가

- 음식의 이름과 열량을 입력하여 음식을 추가한다. 그리고 해당 음식을 식단에 추가할 수 있다.

(3) 음식 삭제

- X 버튼을 눌러 음식을 삭제 시킨다. 사용자의 실수를 방지하기 위해 삭제 여부를 물어본다.

확인 버튼을 누르면 변경 내역이 반영되고 취소 버튼을 누르면 해당 변경 내역은 취소된다.

2026년 9월

일	월	화	수	목	금	토
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			



2026-09-26 (토) 식단표

아침

잡곡밥 · 55 kcal
 시금치 · 45 kcal
 소고기 국 · 423 kcal

점심

잡곡밥 · 55 kcal
 잡곡밥 · 55 kcal

저녁

시금치 · 45 kcal
 시금치 · 45 kcal
 시금치 · 45 kcal
 시금치 · 45 kcal
 소고기 국 · 423 kcal
 소고기 국 · 423 kcal
 - - - - -

야식

잡곡밥 · 55 kcal
 시금치 · 45 kcal
 소고기 국 · 423 kcal

총 섭취 열량: 2650 kcal 체중: 66 kg

확인

메인으로 돌아가기

3. 식단 조회

(1) 날짜 선택

- 현재 날짜로부터 1년 전까지의 날짜를 선택하여 해당 식단 표를 확인할 수 있다.

(2) 식단표

- 해당 날짜의 식단 및 총 섭취 열량, 체중을 알 수 있다.

날짜	2026-09-26 (토)	식단 조회	식단 관리
식 단 표			
아침 잡곡밥 · 55 kcal 시금치 · 45 kcal 소고기 국 · 423 kcal		점심 잡곡밥 · 55 kcal 잡곡밥 · 55 kcal	
저녁 시금치 · 45 kcal 시금치 · 45 kcal 시금치 · 45 kcal 시금치 · 45 kcal 시금치 · 45 kcal		야식 잡곡밥 · 55 kcal 시금치 · 45 kcal 소고기 국 · 423 kcal	
총 섭취 열량: 2650 kcal 체중: 66 kg			
최근 체중 변화			
2026-09-19 (7일전) / 55(Kg) 2026-09-26 (오늘) / 66(Kg) ▲11(kg)▲			
목표(kcal)		확인	현재 목표: 3000 kcal
목표 달성 성공! (2650 / 3000 kcal)			

4. 최근 체중 경향, 목표 달성 여부

(1) 메인 페이지

- 현재 이 페이지가 프로그램 실행 할 때 먼저 나오는 페이지로 현재 날짜를 조회하여 식단 리포트를 반영한다.

(2) 최근 체중 변화

- 현재 날짜와 1주 전 사이의 식단이 2개 이상 존재하는 경우 그 두 날짜의 체중을 매칭하여 체중 변화를 볼 수 있다.

(3) 목표 열량 설정

- 사용자가 열량을 입력하여 목표 칼로리를 변경할 수 있다.

(4) 목표 달성 여부

- 목표 열량과 조회한 날짜의 총 섭취 열량을 비교하여 목표 열량 이하인 경우 성공, 초과 시 실패로 표시한다.

```
a = tk.TK()  
b = tk.Label(a, text = "11")  
C = tk.Button(b, command = ...)
```



```
class Widget:  
class Page:  
class Window:  
class GUI:
```

[Cascade GUI Wrapping]

1. 개요

단순히 Tkinter 객체를 무분별하게 사용하는 경우 코드의 설계가 거미줄 처럼 꼬이는 현상이 발생하고 혹여나 앱 사용자가 GUI에 의도치 않은 부하를 안기는 경우가 존재하여 사전에 이를 방지 하기 위해서 객체를 계층 별로 관리하기 위해 해당 기술을 만들었다.

개발자는 이 설계를 만드는데 무려 9일이 걸렸다. (사유 : 하드코딩, 400줄 돌파)

```
class Widget:  
    pass  
  
class Page:  
    pass  
  
class Window:  
    pass  
  
class GUI:  
    def __init__(self):  
        ...
```

[Cascade GUI Wrapping]

2. 설계

이 기술은 Tkinter 객체를 관리하는 클래스인 Widget, Page, Window 클래스와 해당 관리 클래스들의 객체를 정의할 GUI 클래스로 구성되어 있다.

Window (Page 관리)

Page (Widget 관리)

Widget (자신 및 Widget 관리)

GUI (위 클래스의 배치 및 정의)

[Cascade GUI Wrapping]

```
class Widget:
    def __init__(self, ...):
        self._name = ...
        self._parent = ...
        self._obj = tk.Label(...)
        self._sync = ...
        self._event = ...

        self.childWidgets = {}
        self.activated = false

    def syncWidget(self):
        pass
```

3. 주요 내용 (1)

객체 관리 클래스들은 각각 참조할 GUI 객체와 이름 그리고 관리 객체의 디렉터리를 멤버로 가지며 Widget 클래스의 경우 가장 하위 클래스로 해당 클래스의 GUI 객체의 동기화 함수와 이벤트 함수를 가진다.

상위 객체가 동기화 함수를 사용하는 경우 나머지 자식의 동기화 함수도 계층적으로 자동으로 적용되고 이벤트 함수를 발동하는 경우 해당 함수가 끝날 때 까지 호출되지 않게 설계를 하여 GUI에 안정성을 보장한다.

[Cascade GUI Wrapping]

```
class Widget:
```

```
    @property # Read Only
    def parent(self):
        return self._parent
```

```
    @property
    def sync(self):
        return self._sync
```

```
    @sync.setter
    def sync(self, new_sync):
        self._sync = new_sync
        self._obj.config(...)
```

4. 주요 내용 (2)

대부분의 객체 관리 클래스들은 멤버 변수의 값을 바꾸는 경우 해당 멤버 변수에 맞게 클래스가 참조하고 있는 GUI 객체의 구성도 바뀌게 된다. 다만 일부 변수는 읽기 전용으로 구조상으로 변경해서 안되는 값을 가진다.

청춘 일상 다이어트 총력전

< 4. 자체 평가 및 감상 >

(1) 좋은 점 및 아쉬운 점

이 프로젝트를 통하여 객체 지향 프로그래밍이 어떻게 쓰이는지 어느 정도 되어 설계 능력이 더욱 향상 되었다는 점이다.

다만 아쉬운 점은 계층 관리 클래스를 상속이 가능한 형태로 가공하여 자식 클래스로 정의를 했으면 코드줄이 그보다 짧아 졌을 것이라 생각하지만 시간이 촉박하여서 이 부분까지는 구현을 못했다는 점이 아쉽다. 또한 event 연결의 경우 버그가 존재하여 그 부분은 시를 사용하여 해결했다는 점에서 내가 전부 못 만들었다는 것이 아쉽다.

청춘 일상 다이어트 총력전

< 4. 자체 평가 및 감상 >

(2) 총평

예전에는 AI 없이 프로그램 개발을 하고 2학년 1학기 때 데이터베이스 기간 프로젝트로 처음 AI에게 모든 것을 맡겼을 때는 코드가 뒤죽박죽에 버그가 많았으나, 지금은 AI가 내 설계에 따라 코드를 짜줘서 버그가 많이 줄어들고 성능도 대폭 상승하였다. 결국은 설계에 심혈을 기울여 구성을 해야만 AI도 이에 따른 성능 상승이 이루어 진다는 사실을 알았다. 시니어 개발자가 왜 AI를 계속 쓰는지를 알게 되는 것은 덤이다.

감사합니다!